Project Report: 2048 Web Game

Title: Implementation of 2048 Web Game Using Flask, JavaScript, and CSS

Abstract:
The 2048 game is a single-player sliding block puzzle that challenges the player to combine numbered tiles until reaching the 2048 tile. This project implements a web-based version of 2048 using Flask for the backend and JavaScript, HTML, and CSS for the frontend. The application provides real-time interaction, score tracking, and a user-friendly interface with a modern purple-themed design.

---

1. Introduction:
The game 2048 has gained popularity due to its simplicity and addictive gameplay. The aim of the project is to recreate the game in a web environment, providing users with an interactive interface and responsive controls. This project emphasizes the use of web technologies and demonstrates basic concepts of game logic, state management, and frontend-backend communication.

Objectives:

- Recreate the classic 2048 game for web platforms.
- Implement score tracking and game over detection.
- Develop an aesthetically pleasing UI using CSS with a purple theme.
- Provide functionality for restarting the game.

---

2. System Design:
Architecture:

- **Frontend:** HTML, CSS, JavaScript.
    - HTML structures the game board and buttons.
    - CSS applies a modern, purple-themed design with responsive tiles.
    - JavaScript handles keyboard input, dynamic board rendering, and AJAX calls to the backend.

- **Backend:** Flask (Python).
    - Manages game state (grid and score).
    - Provides endpoints for retrieving board data, making moves, and starting a new game.

Data Flow:

1. User presses arrow keys to make a move.
2. JavaScript captures input and sends a POST request to Flask /move endpoint.
3. Flask processes the move, updates the grid, merges tiles, updates score, and adds a new tile.
4. Flask returns updated game state as JSON.
5. JavaScript renders the updated grid and score dynamically.

Game Logic:

- **Grid Representation:** A 4x4 matrix with integer values.
- **Tile Movement:** Functions to move left, right, up, down.
- **Merge Logic:** Adjacent tiles with equal values are merged, increasing the score.
- **New Tile Generation:** After each move, a 2 or 4 tile is added to a random empty cell.
- **Game Over Detection:** Checks for empty cells or possible merges horizontally/vertically.

---

3. Implementation:
Backend (Flask):

- app.py handles routes /, /board, /move, /newgame.
- Functions include move_left, move_right, move_up, move_down, merge, compress, add_new_tile, and game_over.
- Maintains global score and grid.

Frontend:

- **index.html:** Contains board div, score display, new game button, and game over overlay.
- **script.js:** Captures keyboard input, communicates with Flask endpoints, renders board dynamically, handles game over and restart events.
- **style.css:** Provides a purple-themed UI, responsive tile colors based on values, styled buttons, and overlays.

Tile Color Theme:

- Low values: lighter colors (light purple, blue, green)
- High values: darker or distinct colors (purple, teal, gold)
- Active and interactive buttons: bright purple shades for visibility.

---

4. Challenges:

- **Merge Logic:** Ensuring correct merging behavior without skipping tiles or double-counting score.
- **Game Over Detection:** Accurately determining when no moves are possible.
- **Frontend Synchronization:** Keeping frontend display synchronized with backend state for smooth user experience.
- **Dynamic Styling:** Managing multiple tile colors for various values.

Solutions:

- Implemented compress and merge functions separately.
- Added explicit checks for game over conditions.
- Used AJAX fetch calls for asynchronous updates to the board.
- Applied CSS classes dynamically for tile values.

---

5. Results:

- Fully functional 2048 game playable in a browser.
- Real-time score updating and responsive tile movements.

- Game over detection triggers overlay with final score.
- Restart and new game buttons function correctly.
- Modern and visually appealing purple-themed UI.

Screenshots:

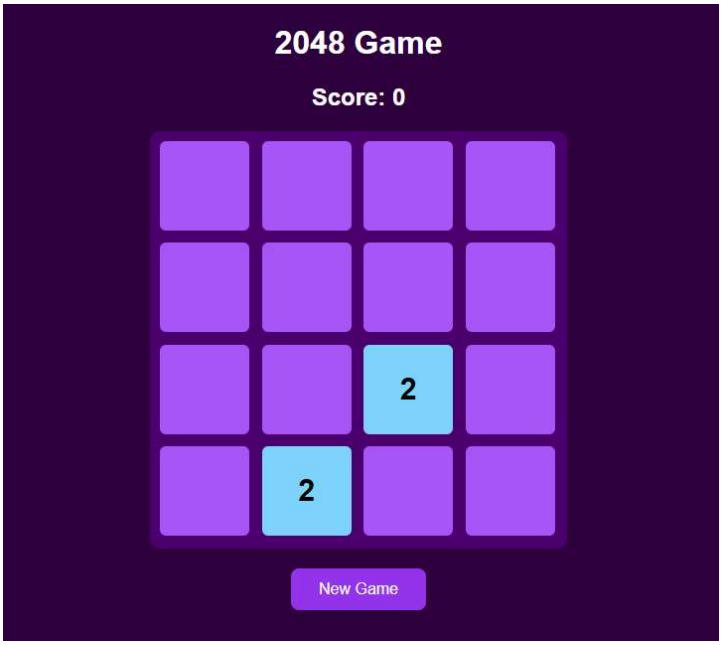

Figure: New Game

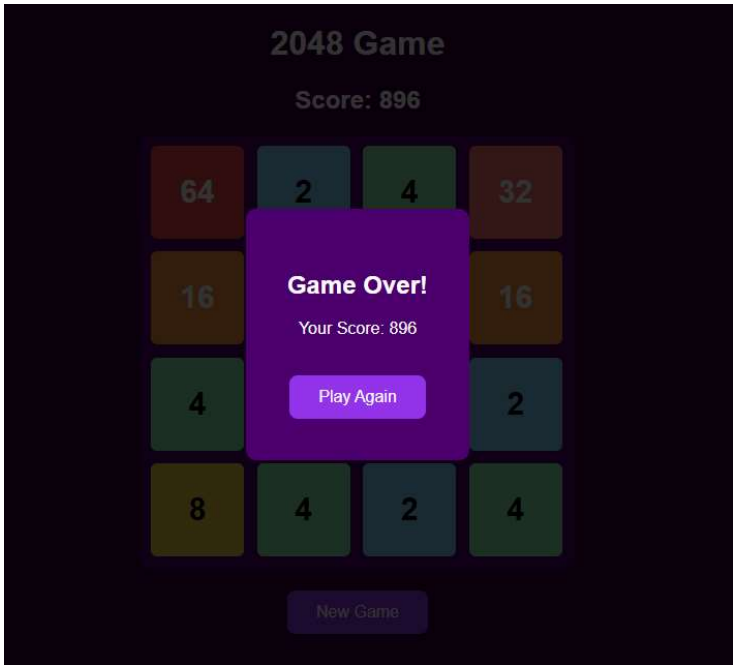

Figure: Game Over

6. Conclusion:
The project successfully implements a web version of the 2048 game, demonstrating key concepts of game logic, frontend-backend communication, and user interface design. The project highlights the integration of Python Flask with JavaScript and CSS for interactive web applications. Future enhancements could include mobile responsiveness, animations for tile movements, and user login to save high scores.

References:

1. 2048 Game Official: https://play2048.co/
2. Flask Documentation: https://flask.palletsprojects.com/
3. JavaScript Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
4. CSS Grid Layout: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Grid_Layout